

EN UNA FRASE Construyes los cinco endpoints de una API RESTful sobre `/libros` y devuelves en cada uno el código de estado correcto (200, 201, 204, 404) con `ResponseEntity`, `@PathVariable` y `@RequestBody`.

LO ESENCIAL

CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre un recurso
GET	Leer; nunca modifica el estado del servidor. Devuelve 200 (o 404 si el recurso no existe)
POST	Crear un recurso nuevo; devuelve 201 Created
PUT	Reemplazar un recurso completo; devuelve 200 OK o 404
DELETE	Eliminar un recurso; devuelve 204 No Content o 404
<code>ResponseEntity<T></code>	Envoltorio de la respuesta HTTP: cuerpo, código de estado y cabeceras. Sin cuerpo, <code><Void></code>
<code>ResponseEntity.ok(body)</code>	200 OK con cuerpo
<code>ResponseEntity.status(HttpStatus.CREATED).body(b)</code>	201 Created con cuerpo
<code>ResponseEntity.noContent().build()</code>	204 No Content sin cuerpo
<code>ResponseEntity.notFound().build()</code>	404 Not Found sin cuerpo
<code>@PathVariable</code>	Extrae el segmento <code>{id}</code> de la URL y lo convierte al tipo declarado; si no puede (<code>/libros/abc</code>), Spring responde 400
<code>@RequestBody</code>	Deserializa el cuerpo JSON al objeto Java (Jackson); necesita constructor vacío y getters/setters. JSON mal formado: 400
<code>@PutMapping("/{id}")</code> / <code>@DeleteMapping("/{id}")</code>	Mapean peticiones PUT y DELETE a un método del controlador
Devolver <code>List<Libro></code> directamente	Válido cuando el código siempre es 200; usa <code>ResponseEntity</code> solo si el código puede variar

ERRORES TÍPICOS

1 POST /libros responde 200 en lugar de 201

✓ el método devuelve `Libro` directamente; envuélvelo en `ResponseEntity.status(HttpStatus.CREATED).body(creado)`.

2 DELETE responde 200 con un texto en el cuerpo

✓ usa `ResponseEntity<Void>` con `noContent().build()` si se eliminó y `notFound().build()` si no existía.

3 @PathVariable String id y un Long.parseLong(id) a mano

✓ declara el parámetro como `Long`: Spring convierte el segmento y responde 400 si no es numérico.

4 El libro creado llega con todos los campos a null o 0

✓ falta `@RequestBody` en el parámetro; sin él, Spring no lee el cuerpo JSON.

5 GET /libros/buscar da 400 al convertir "buscar" a Long

✓ la ruta literal la captura `{id}`. En Spring Boot 3.x las rutas literales tienen siempre prioridad; en versiones anteriores, declara primero las literales o llévalas a otro controlador.

EN EL PROYECTO BIBLIOTECA

`LibroController` pasa de un único `GET /libros` al CRUD completo: `GET /libros/{id}`, `POST /libros`, `PUT /libros/{id}` y `DELETE /libros/{id}`, todos con `ResponseEntity`. `RepositorioLibros` y `BibliotecaService` ganan `actualizar()` (el servicio delega en el repositorio y solo `registrar` escribe en el log, a nivel INFO). `pruebas.http` incorpora las peticiones de los cinco endpoints.

■ CÓDIGO MÍNIMO Y PETICIONES

El patrón `Optional` → 200 o 404, y la creación con 201 (`LibroController`, con `@RequestMapping("/libros")`).

```
@GetMapping("/{id}")
public ResponseEntity<Libro> obtenerPorId(@PathVariable Long id) {
    return service.buscarPorId(id)
        .map(ResponseEntity::ok)           // existe → 200 con el libro
        .orElse(ResponseEntity.notFound().build()); // no existe → 404 sin cuerpo
}

@PostMapping
public ResponseEntity<Libro> crear(@RequestBody Libro libro) {
    Libro creado = service.registrar(libro);
    return ResponseEntity.status(HttpStatus.CREATED).body(creado);
}
```

JAVA

Eliminar responde 204 o 404. `PUT /libros/{id}` reutiliza el patrón del `GET /{id}` con `service.actualizar(id, libro)`.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    if (service.eliminar(id)) {
        return ResponseEntity.noContent().build(); // 204
    }
    return ResponseEntity.notFound().build();      // 404
}
```

JAVA

pruebas.http – CRUD completo de libros

GET http://localhost:8080/libros

###

POST http://localhost:8080/libros

Content-Type: application/json

```
{
  "titulo": "Fahrenheit 451",
  "autor": "Ray Bradbury",
  "anio": 1953
}
```

###

PUT http://localhost:8080/libros/1

Content-Type: application/json

```
{
  "titulo": "El Quijote",
  "autor": "Miguel de Cervantes",
  "anio": 1605
}
```

###

DELETE http://localhost:8080/libros/4

HTTP